A technical explanation of FSRS

Spaced repetition stuff

A technical explanation of FSRS

If you want a simple overview first, start here: ABC of FSRS.

Jarrett has his own article, but I believe mine is easier to understand.

In this article, I'll explain how FSRS-6 works. Note that FSRS-6 is available in Anki since version 25.07.

Table of contents

- R, Retrievability
- Intervals
- S, Stability
- Short-term S
- D, Difficulty
- Optimization aka training

R, Retrievability

Let's start with the forgetting curve. In FSRS v3, an exponential function was used. In FSRS v4, the exponential function was replaced by a power function, which provided a better fit to the data. Then, in FSRS-4.5, it was replaced by a different power function which provided an even better fit. It was used in FSRS-5 as well. Then, finally, FSRS-6 added an optimizable parameter (w20) to the curve to make it personalizable for every user:

FSRS v3: $$R = 0.9^{\frac{t}{S}}$$

FSRS v4: $$R = (1 + \frac{t}{9S})^{-1}$$

FSRS-4.5 and 5: $$R = (1 + \frac{19}{81} \cdot \frac{t}{S})^{-0.5}$$

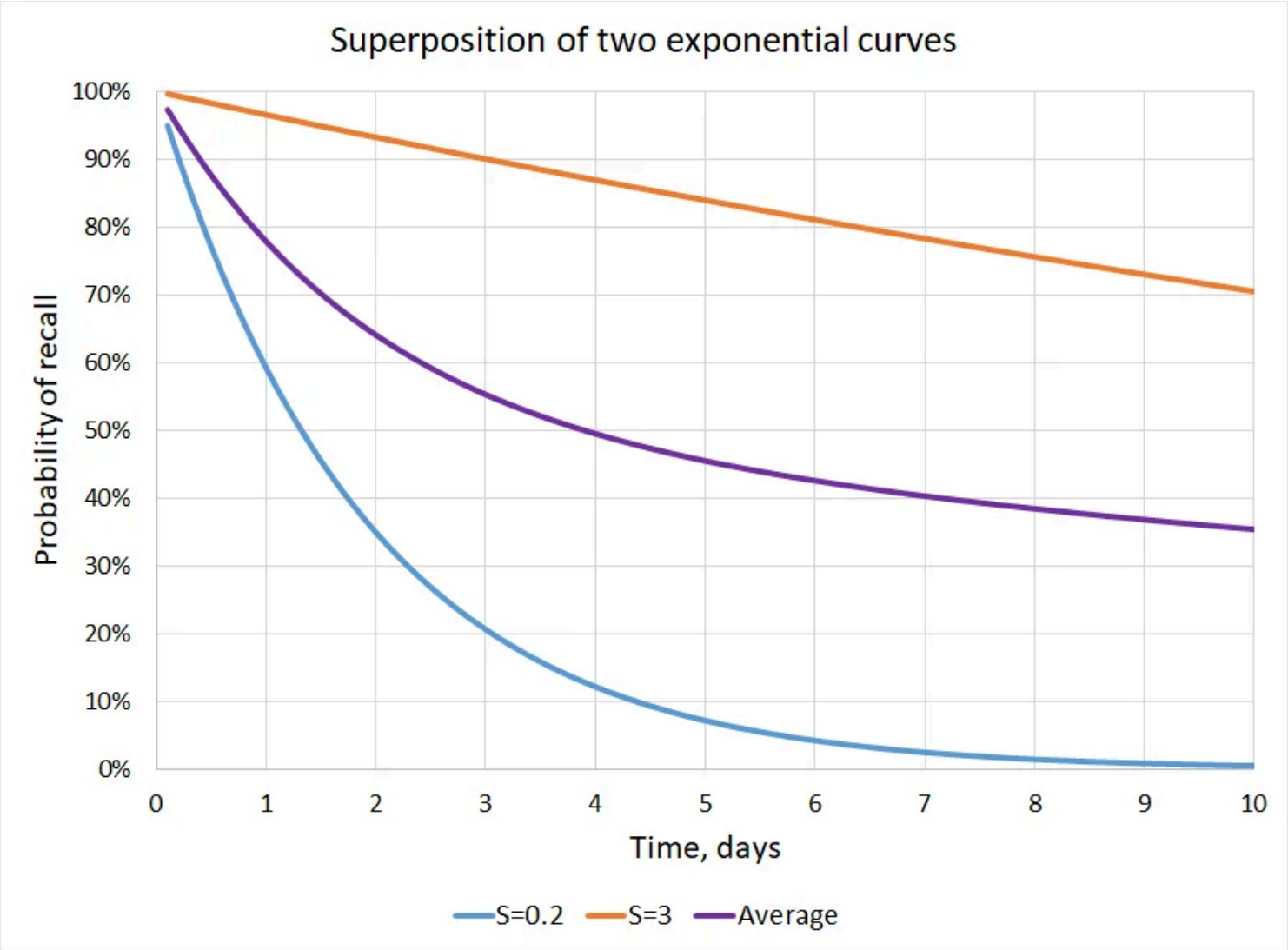
FSRS-6: $$R = (1 + factor \cdot \frac{t}{S})^{-w_{20}}$$

$$factor = 0.9^{-\frac{1}{w_{20}}} - 1$$

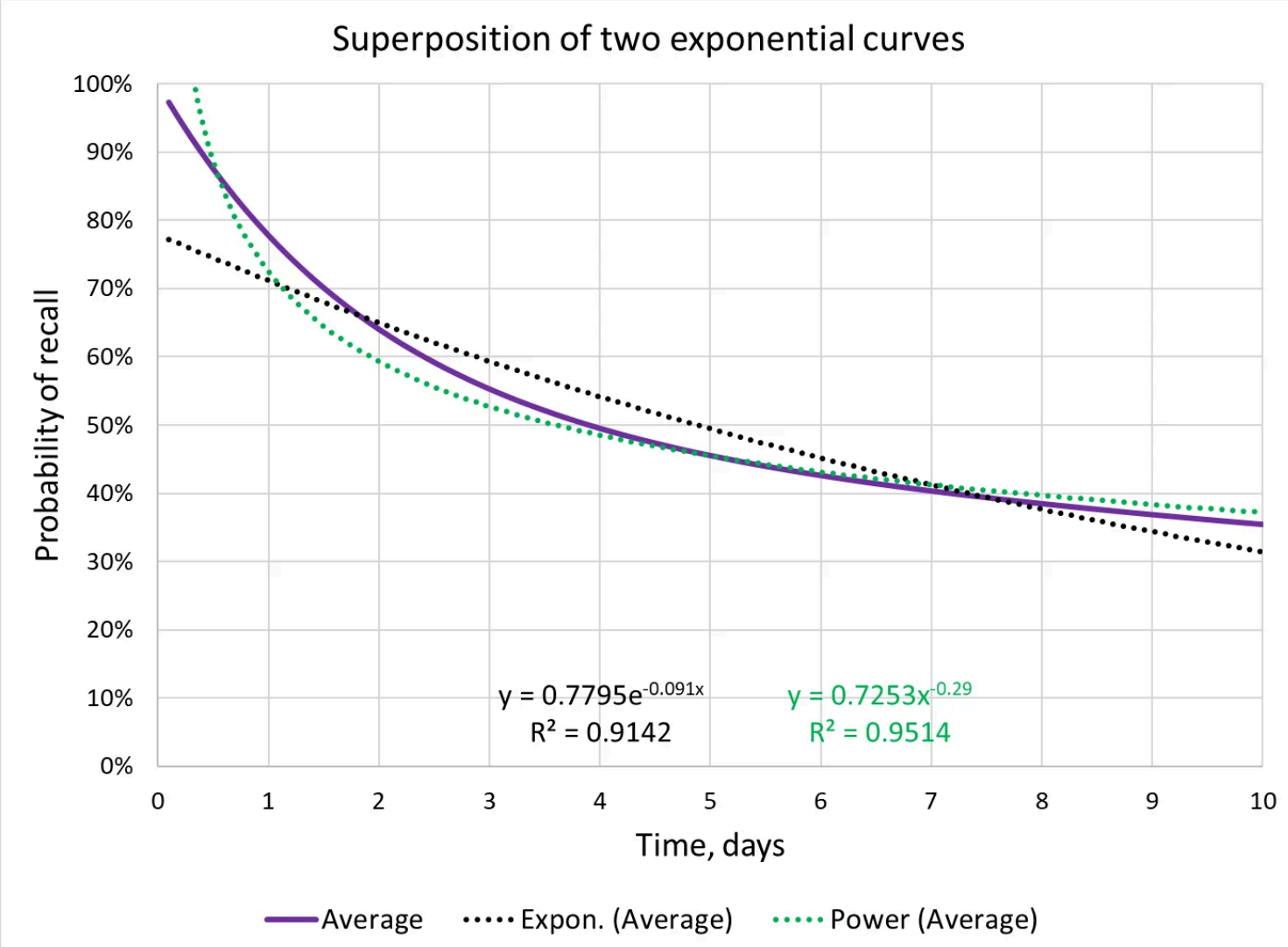
w20 can be between 0.1 and 0.8. For most users it's less than 0.2.

The main difference between these curves is how fast R declines when t>>S. Note that when t=S, R=90% for all four functions. This has to hold true because in FSRS, memory stability is defined as the amount of time required for R to decrease from 100% to 90%. You can play around with them here: https://www.desmos.com/calculator/seqokdyixj.

So why does a power function provide a better fit than the exponential function if forgetting is (in theory) exponential? Let's take two exponential curves, with S=0.2 and S=3. And then let's take the average of the two resulting values of R. We will have 3 functions: R_1=0.9^(t/0.2), R_1=0.9^(t/3) and R=0.5*(0.9^(t/0.2)+0.9^(t/3)).



Now here's the interesting part: if you try to approximate the resulting function (purple), the power approximation would provide a better fit than an exponential one!



Note that I displayed R^2 on the graph, but you can use any other measure to determine the goodness of fit, the conclusion will be the same.

Important takeaway number one: a superposition of two exponential functions is better approximated by a power function.

Intervals

In FSRS-6, intervals are calculated using this formula:

$$I(DR, S) = \frac{S}{0.9^{-\frac{1}{w_{20}}} - 1} \cdot (DR^{-\frac{1}{w_{20}}} - 1)$$

I is interval length, DR is desired retention.

When desired retention is 90%, the interval is equal to stability (before taking into account Anki's fuzz).

S, Stability

Now let's take a look at the main formula of FSRS.

$$S'(D,S,R,G) = S \cdot (1 + w_{15} \cdot w_{16} \cdot e^{w_8} \cdot (11 - D) \cdot S^{-w_9} \cdot (e^{w_{10} \cdot (1-R)} - 1))$$

Here G is grade, it affects w15 and w16. R refers to retrievability at the time of the review.

Let's simplify this formula as much as possible.

$$S'(D,S,R) = S \cdot SInc$$

The new value of memory stability is equal to the previous value multiplied by some factor, which we will call SInc. SInc>=1, in other words, **memory stability cannot decrease if the review was successful**. Easy, Good and Hard all count as "success", Again counts as a memory "lapse". That's why you shouldn't use Hard as a failing grade, only as a passing grade.

SInc is equal to one plus the product of functions of three components of memory (I'll remove the part that depends on the grade for now).

$$SInc = 1 + f(D) \cdot f(S) \cdot f(R)$$

Now let's "unfold" each of them, starting with *f(D)*.

$$SInc = 1 + (11 - D) \cdot f(S) \cdot f(R)$$

Important takeaway number two: the larger the value of D, the smaller the SInc value. This means that the increase in memory stability for difficult material is smaller than for easy material.

Next, let's unfold *f(S)*.

$$SInc = 1 + (11 - D) \cdot S^{-w_9} \cdot f(R)$$

Important takeaway number three: the larger the value of S, the smaller the SInc value. This means that the higher the stability of the memory, the harder it becomes to make the memory even more stable. Memory stability tends to saturate.

This will likely surprise a lot of people, but the data supports it.

Finally, let's unfold *f(R)*.

$$SInc = 1 + (11 - D) \cdot S^{-w_9} \cdot (e^{w_{10} \cdot (1-R)} - 1)$$

Important takeaway number four: the smaller the value of R, the larger the SInc value. This means that the best time to review your material is when you almost forgot it (provided that you succeeded in recalling it).

If that sounds counter-intuitive, imagine if the opposite were true. Imagine that the best time to review your material is when you know it perfectly (R is very close to 100%). There is no reason to review something if you know it perfectly, so this can't be true.

Last but not least, we need to add three more parameters: one to control the overall "scale" of the product, and two more to account for the user pressing "Hard" or "Easy". w15 is equal to 1 if the grade is "Good" or "Easy", and <1 if the grade is "Hard". w16 is equal to 1 if the grade is "Hard" or "Good", and >1 if the grade is "Easy". In the current implementation of FSRS, 0<w15<1 and 1<w16<6.

$$SInc = 1 + w_{15} \cdot w_{16} \cdot e^{w_8} \cdot (11 - D) \cdot S^{-w_9} \cdot (e^{w_{10} \cdot (1-R)} - 1)$$

Now all we have to do is just take the previous value of S and multiply it by SInc to obtain the new value. Hopefully that formula makes more sense to you now.

The formula for the next value of S is different if the user pressed "Again".

$$S'(D, S, R) = min(w_{11} \cdot D^{-w_{12}} \cdot ((S + 1)^{w_{13}} - 1) \cdot e^{w_{14} \cdot (1-R)}, S)$$

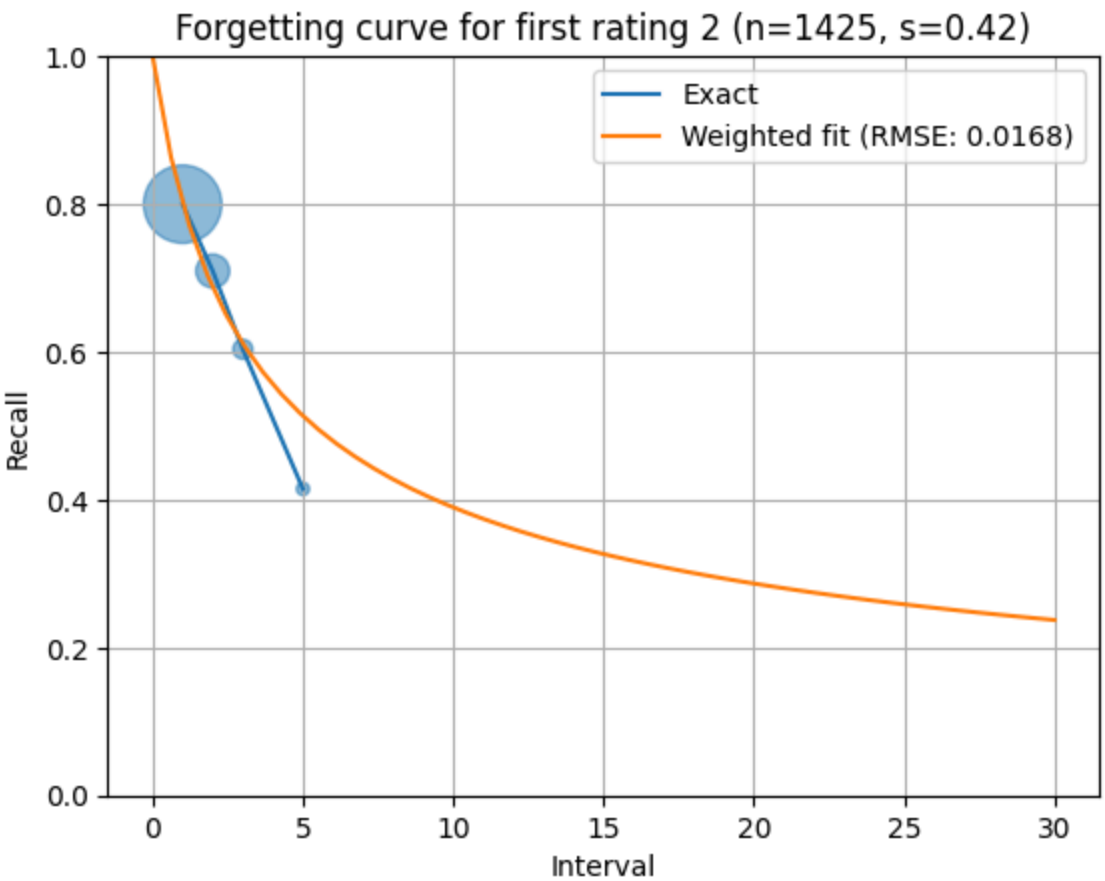
min(..., S) is necessary to ensure that post-lapse stability can never be greater than stability before the lapse. w11 serves a similar purpose to e^w8 in the main formula: it just scales the whole product by some factor to provide a better fit.

An interesting detail: in the main formula, the function of D is linear: f(D)=(11-D). Here, however, *f(D)* is nonlinear. Me and LMSherlock have tried different formulas, and surprisingly, these provide the best fit.

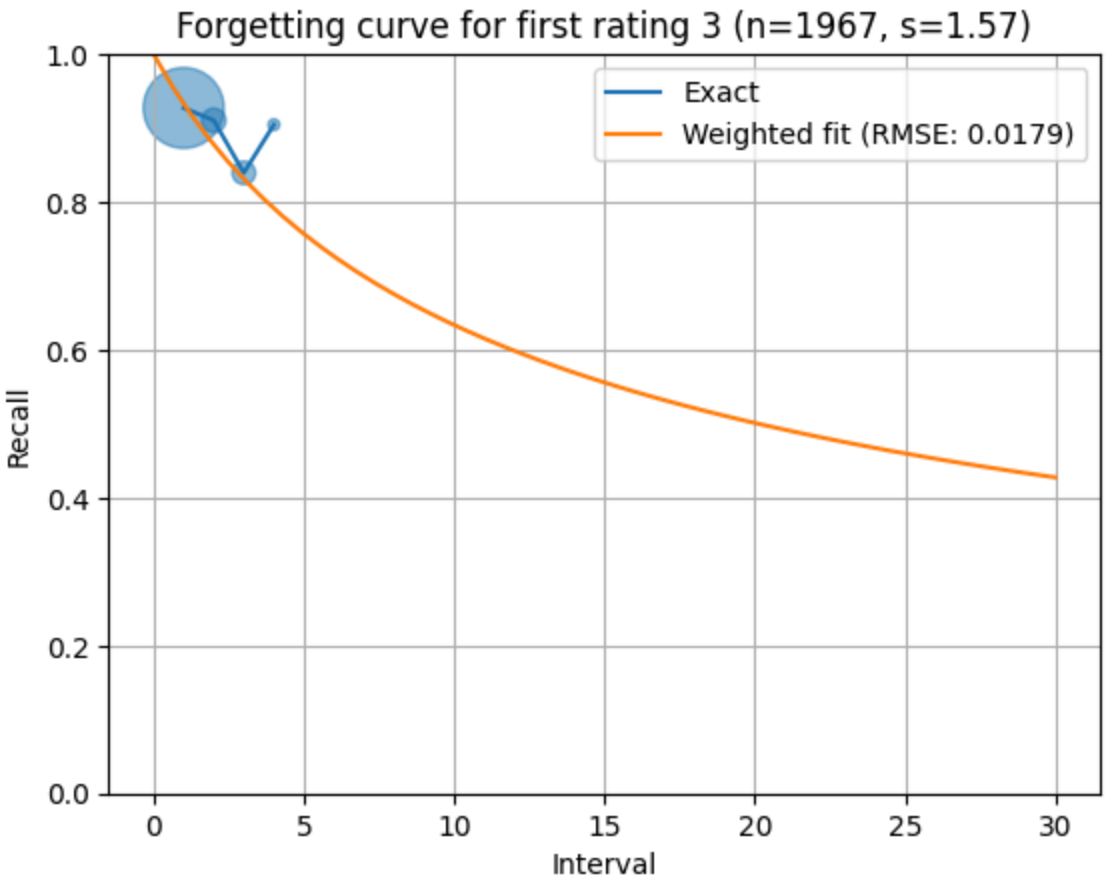
There is one problem with these formulas, though. Since both formulas require the previous value of S to calculate the next value of S, they cannot be used to estimate initial stability after the first review since there is no such thing as a "zeroth review". So initial stability has to be estimated in a completely different way.

Here's how. First, all reviews are grouped into 4 groups based on the first grade (Again, Hard, Good, Easy). Next, intervals and outcomes of the second review are used to plot graphs like the ones below.

This is for first grade="Hard":



This is for first grade="Good":



(pretend that it says "Retention" and not "Recall")

On the x axis, we have t, the interval length. On the y axis, we have the proportion of cards that the user got right for that particular interval aka retention. The size of the blue circle indicates the number of reviews. A bigger circle means more reviews.

For example, say we want to find the initial S for the "Good" grade. So we find all cards where the first grade is "Good", and calculate retention after a one day interval, retention after a two-day interval, retention after a three-day interval, etc. That way we get retention at different interval lengths, with the interval on the X axis and retention on the Y axis.

Next, we need to find a forgetting curve that provides the best fit to this data, in other words, we need to find the value of S that minimizes the difference between the measured probability of recalling a card after this many days and the predicted probability. This is done using a fast curve-fitting method, so it only takes a fraction of the overall time required to optimize parameters. I could write three or four more paragraphs about the specifics of this curve-fitting procedure, but that's neither interesting nor very important for understanding FSRS as a whole.

The first four parameters that you see in the "FSRS parameters" window are the initial stability values.

FSRS parameters

0.1033, 0.1033, 1.0978, 20.3628 6.4315, 0.6296, 2.4602, 0.0014, 1.3171, 0.1323, 1.0180, 1.6567, 0.0968, 0.2038, 1.9805, 0.2279, 4.2558, 0.9442, 0.6892, 0.2048, 0.1201

Short-term S

This was introduced in FSRS-5. While neither FSRS-5 nor FSRS-6 have a proper model of short-term memory, they have a crude heuristic. In FSRS-6 the new S after a same-day review (what counts as "same-day" depends on Anki settings) is calculated using this formula:

$$S' = S \cdot e^{w_{17} \cdot (G - 3 + w_{18})} \cdot S^{-w_{19}}$$

w17 and w18 control how much grades affect S. w19 plays a role similar to w9 above: as S increases, the change in S decreases. For small values of S, the impact of same-day reviews is greater.

Additionally, there is an extra check to ensure that S' >= S if G >= 3. In other words, Good and Easy cannot decrease S, Hard and Again can. This is different from the main formula, where Hard cannot decrease S.

D, Difficulty

Unlike S and R, D has no precise definition and is just a crude heuristic. Here is the formula for initial D, after the first review.

$$D_0(G) = w_4 - e^{w_5 \cdot (G-1)} + 1$$

G is grade. Again=1, Hard=2, Good=3, Easy=4. We have tried turning these four values into optimizable parameters (as opposed to just using constants), but that didn't improve accuracy. Note that the output of this formula is clamped afterwards to ensure that D0 is between 1 and 10.

The formula for the next value of D is more complicated.

First, we calculate the change in difficulty that only depends on the grade:

$$\Delta D = -w_6 \cdot (G - 3)$$

Then we modify it using what I call "linear damping". Linear damping makes it so that as difficulty approaches 10 (maximum value), each update gets smaller and smaller. That way D is never *exactly* 10, just asymptotically approaching 10.

$$D' = D + \Delta D \cdot \frac{10 - D}{9}$$

Finally, we apply what LMSherlock calls "mean reversion", where the current value of D is slightly reverted back to the default value that corresponds to the Easy button: w4.

$$D'' = w_7 \cdot D_0(4) + (1 - w_7) \cdot D'$$

I could write the entire formula without splitting it into several parts (like D' and D"), but I think it's easier to understand this way.

This means that if you keep pressing "Good", difficulty will eventually converge to its default value, which is an optimizable parameter.

Putting the "mean reversion" aside, difficulty basically works like this:

Again = add a lot
Hard = add a little bit
Good = nothing
Easy = subtract a little bit

Again and Hard increase difficulty, Good doesn't change it (again, before "mean reversion" is applied), and Easy decreases it. We've tried other approaches, such as "Good = add a little bit", but nothing improved the accuracy.

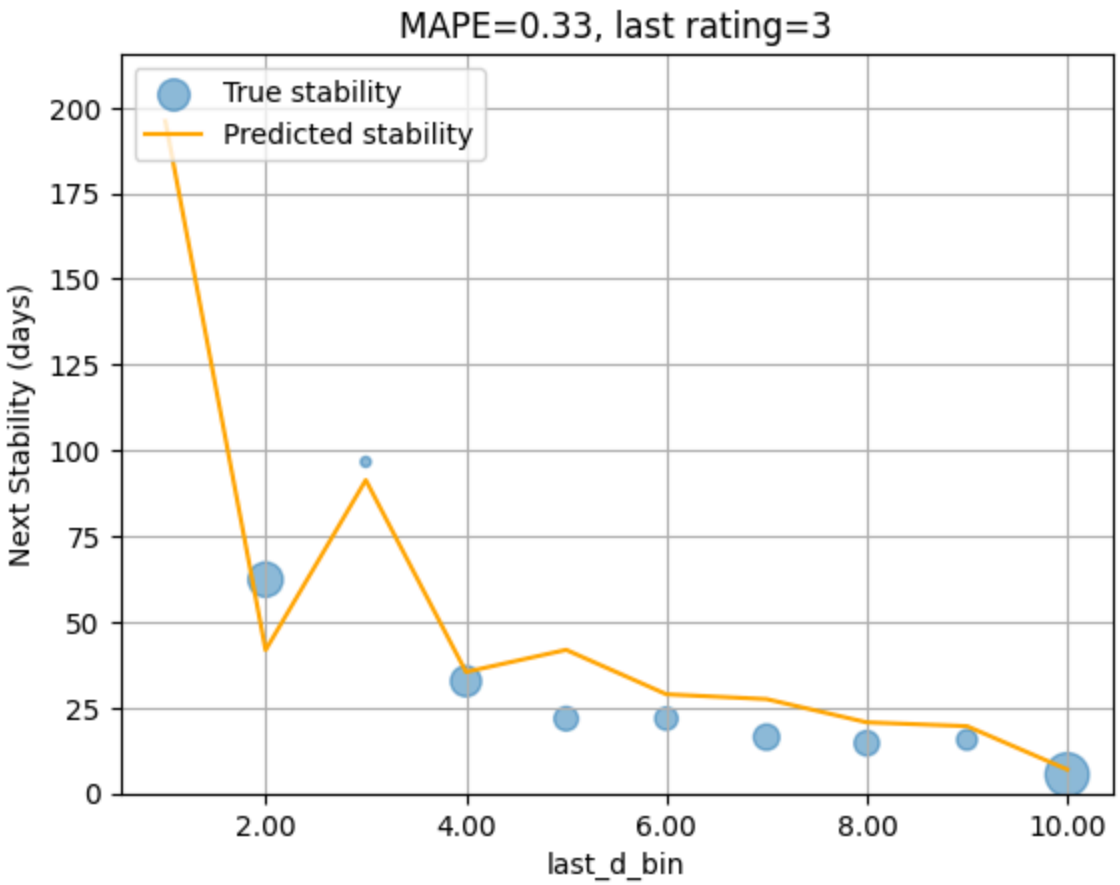
The current definition of D is flawed: it doesn't take R into account. Imagine two situations:

You pressed "Good" when the probability of recalling this card was 90.00%.
You pressed "Good" when the probability of recalling this card was 0.01%.

Clearly, these two situations are different, because in the second one it's very surprising that you recalled a card when R was so low, whereas in the first situation it's not surprising. In the latter case, difficulty should be adjusted by a much greater value than in the first case.

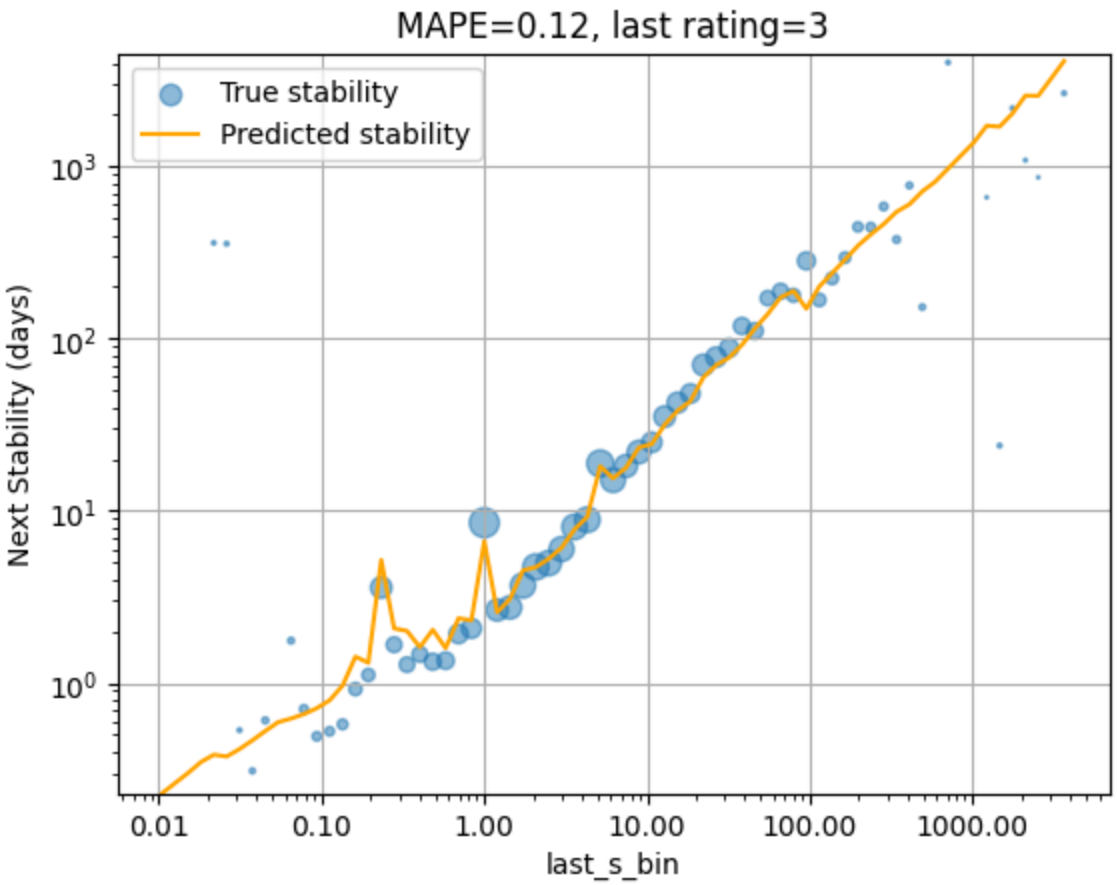
Important takeaway number five: properly defined difficulty must depend on retrievability, not only on grades.

However, a more in-depth analysis reveals that the current formula works reasonably well.



On the x axis, we have D, and on the y axis, we have predicted and measured S. Blue dots are values of memory stability that have been measured from my review history, and the orange line is the predicted value of memory stability. Of course, both are *averages* that require thousands of reviews to estimate accurately.

As you can see, the orange line is close to the blue dots, meaning that, *on average*, predicted stability is close to actual stability. Though the fit is worse for low values of D, they are also based on fewer reviews. This is based on one of my own decks. Also, I say "close", but mean absolute percentage error (MAPE) is around 33% for my collection here, meaning that, on average, FSRS is off my 33% when predicting the value of S. Note that this depends on what you have on the X axis. For example, below is a similar graph, but for S as a function of it's own previous value. Here, MAPE is 12%. Also, both graphs are specifically for when the user presses "Good".



Side note: D ranges from 1 to 10, but in the built-in version of FSRS, D is displayed as a number between 0 and 1. This conversion is completely unnecessary in my opinion.

It's important to mention that me and Sherlock have tried to incorporate R into the formulas for D, but it didn't improve the accuracy. Even though we know that in theory D should depend on R, we don't know how to actually add R to D in a way that is useful.

Optimization aka training

I won't go too into detail about this, instead you can watch this video about gradient descent by 3blue1brown or this one (the second one is better IMO). The short version:

1. Choose some initial values for all parameters (except the first four in our case, since they are estimated before the "main" optimization procedure).
2. Change them by some small number.
3. Check how much the loss function has changed. Since our case is effectively a binary classification problem (each review is either a "success" or a "lapse"), log-loss aka binary cross-entropy is used. The loss function measures the "distance" (in some mathematical sense) between predictions and real data, and the choice of the loss function depends on the task.

4. Update parameters to decrease the loss.
5. Keep updating the parameters based on how much it affects the loss (steps 2-4) until the loss stops decreasing, which indicates that you have reached the minimum.

Of course, it's a lot more nuanced than that, but if you want to learn about gradient descent, there are hundreds of videos and articles on the Internet, since this is how almost every machine learning algorithm in the world is trained.

Note that the first four parameters are first estimated directly from the results of first and second reviews, and then are further optimized by gradient descent.

←Return to homepage

This page was generated by GitHub Pages.